



FROM ONTOLOGY TO SYSTEM ARCHITECTURE: AN MBSE APPROACH TOWARD THE REALIZATION OF URBAN AIR MOBILITY

Sonali Sinha Roy¹, Yinchien Huang¹, Dalia Bekdache¹, Tien-Yueh Fung¹, Brady Beck¹, Cesare Guariniello² & Daniel DeLaurentis³

¹Graduate Student, School of Aeronautics and Astronautics, Purdue University, West Lafayette, IN, USA

²Research Scientist, School of Aeronautics and Astronautics, Purdue University, West Lafayette, IN, USA

³Bruce Reese Professor of Aeronautics and Astronautics, Purdue University, West Lafayette, IN, USA

Abstract

Urban Air Mobility (UAM) is an advanced aviation concept realized via a system of systems (SoS) involving a diverse set of interrelated systems. Model-based systems engineering (MBSE) is well-suited for defining the architecture for such a SoS. However, despite some common fundamental traits of the UAM SoS, the specific architecture and operational parameters will vary from one metro area to another. Ad hoc models for UAM architecture and operations in each metro area can lead to inconsistencies, confusion, and eventually operational difficulties. To prevent such an outcome, this paper proposes a structured framework for the elaboration of UAM architectures by leveraging ontology and reference models. These are used to derive metro-area-specific architectures and operational models. Ontology unifies understanding of the systems, relationships, and processes within the UAM SoS. With ontology as a foundation, a reference model for the baseline architecture serves as the template for metro-area-specific architecture models. The combination facilitates rapid generation of specific UAM architectures and use cases, as exemplified in this paper. We summarize why this step toward a fully modular and reusable framework prepared specifically for UAM can accelerate progress toward the realization of this ambitious concept.

Keywords: Urban Air Mobility, Ontology, MBSE, SysML, System Architecture.

1. Introduction

Urban Air Mobility (UAM) is a concept detailing near-future transportation of people throughout urban environments. UAM is derived from Advanced Air Mobility (AAM), a plan focused on developing an integrated transportation network for people and cargo using innovative aviation technologies [1]. UAM operations relate to aerial transportation in densely populated metropolitan areas and their surrounding suburbs [2]. Vehicles operating in the UAM ecosystem may fly between major airports, urban aerodromes or vertiports, and other destinations adjacent to the urban center in an air-taxi service model. UAM contains several systems working in unison to transport people across highly populated zones. Since it combines several independently operating actors to solve a problem of extreme complexity, UAM qualifies as a system of systems (SoS) [3, 4]. Stakeholders and systems within UAM include but are not limited to, the regulators, Providers of Services for UAM (PSU), fleet operators, passenger-carrying autonomous aerial vehicles (generally eVTOL), UAM aerodromes (also known as vertiports), and public-interest stakeholders [5]. The UAM SoS for any particular location may have multiple PSUs that support UAM operators to meet operational requirements while maintaining airspace safety, security, and efficiency. There can also be multiple fleet operators, which are the entities responsible for monitoring eVTOL vehicle operations.

Both NASA [6] and FAA [1] have released their respective versions of UAM Concept of Operations (ConOps). NASA has defined six UAM Maturity Levels (UML) to describe the projected growth of

the UAM concept from initial formulation to a fully-mature ecosystem. Past and ongoing work related to UAM [7, 8, 5], focuses on UML-4 and UML-5, i.e., semi-autonomous or human-in-the-loop operations. Many research efforts are aimed at progressing toward UML-6, or fully autonomous operations at maturity. As the community continues to build toward an integrated UAM ecosystem, it is essential to characterize the architecture that organizes the systems and processes that the UAM SoS encompasses. We illustrate in this paper that this can be achieved using two tools – ontology and model-based systems engineering (MBSE). This paper proposes an ontology-MBSE pipeline to create UAM models, thereby ensuring consistency and stakeholder unanimity in the design, organization, and operations of the UAM SoS.

In the sections that follow, we first introduce the fundamental concepts of ontology and MBSE, then discuss our proposed approach, and finally use three examples to demonstrate this approach.

2. Ontology

Ontology provides a structured framework for organizing knowledge within specific domains. It formalizes the definition of entities, their attributes, and relationships, establishing an unambiguous language for communication and understanding. It is, therefore, a powerful tool, especially for SoS problems. In system engineering/architecting settings, ontology serves several critical purposes. Firstly, it defines classes, properties, and relationships, fostering standardized communication among distinct modelers and stakeholders. Secondly, it facilitates semantic interoperability, ensuring that information can be shared and comprehended across diverse systems or disciplines.

When applied to model-based systems engineering (MBSE), ontology can enrich system modeling by providing context and consistency. It can help create comprehensive models that accurately depict elements and their behaviors within complex systems. This structured representation supports analysis, simulation, and reasoning related to system properties and behaviors. Certain features of ontology make it desirable for use in systems engineering and systems analysis. For example, using a reasoner in conjunction with traditional ontology enables logical deductions, inference, and queries by utilizing the established relationships and rules encoded within the ontology. This in turn helps define the solution space for system architecture design and aids certain types of decision-making. Additionally, incorporating database features allows the storage and retrieval of ontological data, enabling efficient management and manipulation of structured information. Together, they facilitate advanced functionalities such as semantic reasoning, complex querying, knowledge organization, and efficient data management across diverse applications. Some application areas for ontology in literature include avionics and air traffic management [9], aeronautics data fusion [10], and early product development [11]. Ontology can be a valuable tool in SoS design and analysis [12, 13].

I need to pick one, then.

2.1 Ontology for UAM

In UAM, ontology can delineate operational components, regulations, vehicle characteristics, and other pertinent factors essential for designing and managing UAM service. An ontological approach is particularly appropriate for handling the SoS-unique aspects, especially for safety and regulatory compliance. In this paper, we propose a domain ontology for UAM to enable interoperability and communication among multiple disciplines or across different stakeholders. A few possible uses of ontology for UAM applications are listed below.

1. **Integration of information from multiple sources:** In the complex UAM SoS, data from various sources such as weather sensors, air traffic control systems, ground infrastructure, vehicle telemetry, and passenger information systems need to be combined to ensure smooth and efficient operations. Ontology facilitates this integration by defining a common vocabulary and set of properties, enabling different systems to be integrated as a single data source. These structured data can then be queried to perform various UAM activities.
2. **Software system inter-operation:** In UAM, various software systems – from air traffic management and vehicle control systems to passenger service platforms and urban infrastructure

management – need to work together with little to no human supervision. Ontologies enable these disparate systems to share and understand data by offering a unified set of rules for data representation and exchange. This framework ensures that information from different sources is consistently interpreted and processed, reducing the risk of misunderstandings and the cost of software development.

3. **Autonomous operation:** Autonomous UAM systems require sophisticated decision-making capabilities to navigate complex urban landscapes safely and efficiently. Ontologies enable these systems to retrieve and process the vast amounts of data. This structured knowledge base allows autonomous systems to anticipate and react to hazards proactively, adjusting paths and behaviors in real time to avoid collisions and comply with regulations.
4. **Research and Development:** In the rapidly evolving field of UAM, R&D efforts require a deep understanding of diverse and complex concepts. Ontologies facilitate this by offering a clear, formalized representation of knowledge, allowing researchers to identify relationships and dependencies between different components and processes. This structured approach enables efficient data management, hypothesis testing, and simulation. Additionally, by using the ontology graph database, researchers can explore the relations between various elements in multiple domains, which can enable innovative solutions and insights for the UAM industry.

3. Model-based Systems Engineering (MBSE)

According to the Systems Engineering Body of Knowledge [14], Model-based Systems Engineering (MBSE) is a “paradigm that uses formalized representations of systems, known as models, to support and facilitate the performance of Systems Engineering tasks throughout a system’s life cycle.” MBSE is commonly used for system design and analysis, including requirements management, verification, and validation. In contrast to the traditional document-based approach, MBSE uses system models as artifacts to contain and exchange information about the system. MBSE provides a standardized visual representation of system components, relationships, and behaviors. This improves comprehension and communication among stakeholders. Advantages like easy error detection, iterative model improvement, requirement traceability, and model reusability have made MBSE very popular among systems engineers. For the efficient use of MBSE, models are often based on pre-defined metamodels [15, 16] or modeling languages such as the Unified Modeling Language (UML), Systems Modeling Language (SysML), Capella, etc.

3.1 MBSE for System Architecture Design

System architecture is the conceptual definition of the structure, components, modules, interfaces, behaviors, and data for a system to satisfy its specified requirements. System architecture provides a blueprint for the configuration and hierarchy of the components or subsystems, and their interactions within the system. System architecture design is an iterative process that involves collaboration among various stakeholders. It plays a crucial role in ensuring the system is well-organized, fulfills its intended purpose, and can adapt to evolving requirements and technologies. System architectures can be represented using digital models. An MBSE approach is especially valuable for developing domain-specific system architectures. In many applications, there is a baseline architecture or reference architecture [17, 18] which can be customized for specific purposes. In such cases, it is practical to create an abstract model for the reference architecture, i.e., a ‘reference model’, which can be replicated or modified to create unique system architectures for specific use cases.

Reference models allow modelers to prioritize problem-solving by alleviating the intricacies of modeling semantics and language. Reference models can be seen as higher-level abstraction models that describe the common characteristics within a particular domain. They serve as reusable templates that can be customized for various scenarios and use cases within the domain. For example, one may create a reference model for “airport” which includes the common characteristics of all airports, such as runways, hangars, check-in counters, security check areas, seating areas, etc. Then, this

reference model can be used to create a specific model “FLR” to simulate the structure and configuration of Florence Airport in Italy. Using the same reference model, another specific model “LGA” can be created for La Guardia Airport, New York City, USA.

3.2 MBSE for UAM

UAM is a complex concept that involves several different types of systems – vehicles, fleet operators, UAM service providers, vertiports, etc. MBSE is widely being used [19, 20, 5] for the development of advanced aviation concepts such as UAM. Since UAM is a relatively new and developing concept, the system requirements, architecture, scope, etc. are constantly evolving. Well-designed models can provide a visual and systematic medium for representing, analyzing, and communicating the UAM SoS, with the added convenience of flexibility and iterative improvements. A few ways in which MBSE can be used for UAM are listed below:

1. **Conceptual and Behavioral Modeling:** MBSE can be an effective tool for developing conceptual models that represent the high-level structure and behavior of the UAM SoS. This includes identifying and modeling all the constituent systems, their hierarchy, functions, interactions, and dependencies. For example, a Block Definition Diagram (BDD) in SysML can be used to define each type of system as a ‘block’ with specific attributes and behaviors. The ‘Vehicle’ block might have attributes such as model/build, battery level, current trip status, etc. Its behaviors such as signal transmission and object detection are captured through state machines, activity diagrams, or sequence diagrams.
2. **Requirements Definition and Allocation:** The UAM system requirements can be represented in a structured manner within a model. Functional decomposition can help associate requirements with the appropriate systems to ensure traceability. For example, a requirement related to object detection must be assigned to the aircraft.
3. **Interface Definition:** Models can be used to define the input/output interfaces and communication channels between systems, thereby establishing well-defined data flow protocols. For example, the vehicle-to-vehicle link for broadcasting telemetry or sharing warnings must be properly defined in the UAM model.
4. **Documentation and Visualization:** Documentation for the system can be directly generated from the model. This helps in maintaining consistency and comprehensiveness for engineering applications. Visual representations such as block diagrams, deployment diagrams, interface control diagrams, etc. can effectively communicate the system architecture to various stakeholders like policymakers and investors.
5. **Simulation and Analysis:** The model can be leveraged for simulation and analysis to assess the performance, reliability, and other important aspects of the UAM SoS architecture. Examples include the comparison of different architectures and the evaluation of SoS performance under off-nominal conditions.

The use of MBSE for UAM SoS can support effective decision-making, collaboration, and the development of robust and optimized architectures. The actual UAM system design is likely to vary from one metropolitan area to another. Without a unanimous understanding of the requirements, regulations, infrastructure, and operations, each metropolitan area will need to make an ad hoc model for its own UAM system. This can lead to confusion, operational difficulties, and even disastrous accidents. To prevent this, it is crucial to establish a common vocabulary and universal definitions for UAM applications. Therefore, we believe it is beneficial to create a UAM ontology and a reference model for the generic UAM architecture. Specific models can then be derived from the reference model for UAM implementation in various cities. In the next section, we provide an overview and a few examples of our proposed approach.

4. Proposed Approach: UAM Modeling Pipeline

Ontology can be a key enabler of model-based systems engineering. It has been utilized as a means to support and improve MBSE [21, 22, 23]. In this paper, we propose the combined use of ontology and MBSE to support UAM architecture development and operations. Our approach uses ontology to establish a shared set of domain-specific concepts and vocabularies, followed by a reference architecture model that formalizes the structures, relationships, hierarchy, and rules for interactions within the UAM domain. Metro-area-specific models can then be derived from these two artifacts. The following subsections explain our approach of combining UAM ontology with a reference model to generate specific models for different cities and use cases.

4.1 Overview

Our proposed approach of combining ontology with MBSE for UAM is schematically described in Figure 1. The key systems engineering artifacts that factor into this approach are discussed below.

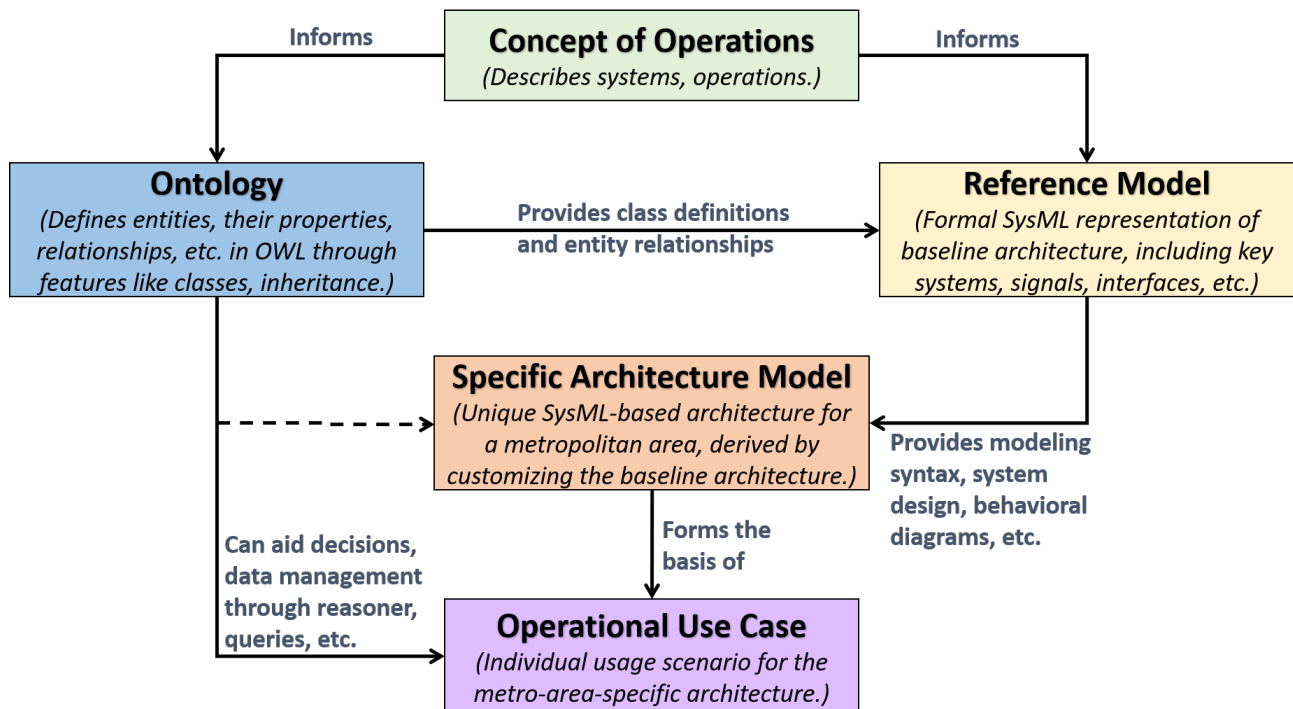


Figure 1 – Proposed modeling approach. Ontology informs the reference model, from which the specific model is derived. Combined use of ontology and MBSE can establish a common vocabulary and a shared understanding of the context, composition, and operations of the UAM SoS.

- **Concept of Operations:** A complex problem domain is first described by a concept of operations. This document outlines the systems involved, their operations, and even some requirements and specifications. The concept of operations informs the ontology and the reference model and is therefore the starting point of our proposed approach. This paper follows NASA's UAM concept of operations [6].
- **Ontology:** Ontology defines the features of and relationships between entities. Different systems, subsystems, signals, messages, etc. can be defined as classes, and their properties can be specified. Communication protocols (e.g., signal types and data structures, valid senders and receivers, etc.) can be established. All these definitions can be transferred to the reference model. Moreover, ontology-based features like reasoners and queries can be directly utilized for additional functionality in the specific architecture and operational models. In this paper, the UAM ontology has been built using Protégé, an ontology development environment based on the Web Ontology Language (OWL). Figure 2 shows the classes and their mutual relationships defined in the ontology model.

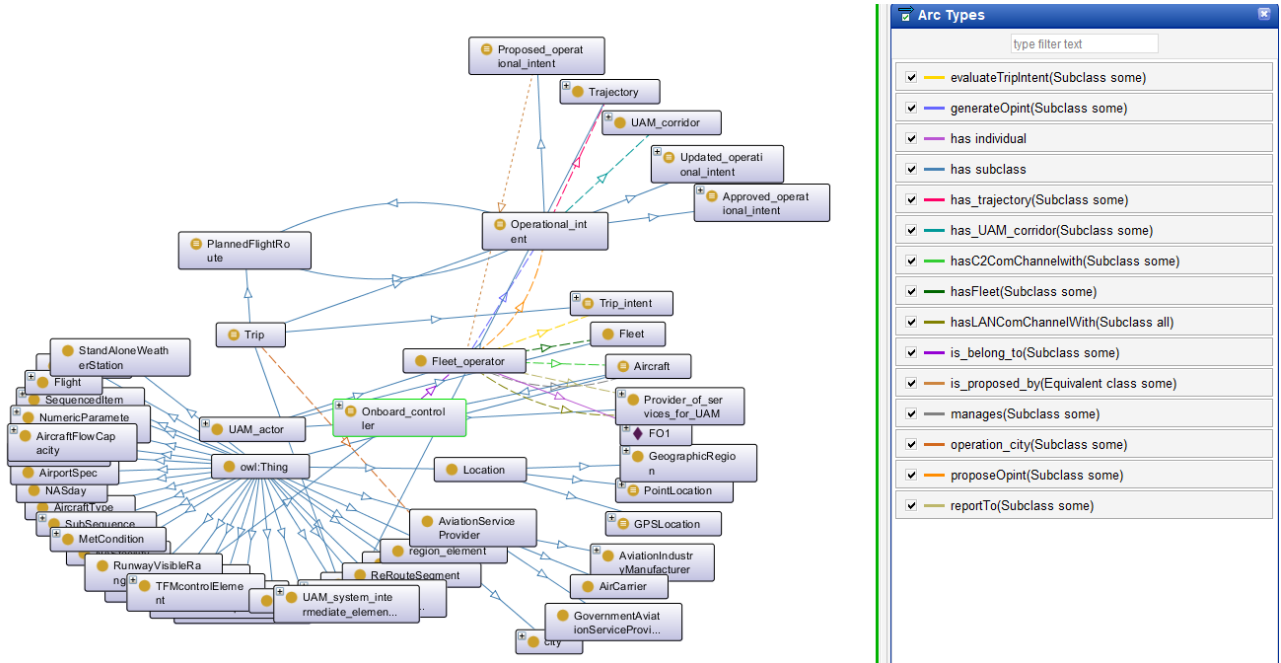


Figure 2 – Visualization of UAM ontology. The graph includes the classes (labeled in yellow) and the color-coded object properties between entities.

- **Reference Model:** The reference model is an abstract model of the UAM SoS that includes the formal representation of all the key systems and relationships. In our case, each of the primary actors (vehicles, fleet operators, etc.), important signals and messages (e.g., telemetry, operation intent), as well as key external actors (e.g., passengers) are defined using a variety of diagrams in the reference model. Our reference model has been created using the Systems Modeling Language (SysML) on Dassault Systèmes MagicDraw. Figure 3 shows the main block definition diagram (BDD) of our UAM SoS reference model.
- **Specific Architecture Models:** To leverage the baseline architecture represented by the reference model, the specific architecture models are created using the same modeling tool, i.e., MagicDraw. The blocks, data types, and diagrams in the reference model can be replicated or customized for specific metropolitan areas to create unique UAM architectures. For example, the layout of the vertiports, number of fleet operators, number and types of vehicles, etc. will be different for different cities. Therefore, their individual UAM architecture, which is a composite of all these factors, will undoubtedly be distinct from each other. These individual architectures can be defined using specific models for each metropolitan area.
- **Operational Use Case Models:** Once the specific UAM architecture for a metropolitan area has been realized, an operational model is needed to manage the day-to-day operations of the UAM SoS. This can be an ABM (agent-based model) in AnyLogic, MASON, or any custom-made program capable of representing all the actors and their behaviors, simulating various use cases, and integrating multiple databases relevant to the UAM SoS. Additionally, the operational use case model can be used for predictive analytics and data-driven decision-making.

4.2 Ontology and MBSE in UAM Development and Operational Stages

A well-defined UAM ontological model forms the basis of our reference architecture. Ontological classes can be redefined in SysML as blocks. The 'object properties' and 'data properties' of an ontological class can be defined as 'value properties' of the corresponding block in SysML. Data types in OWL can be recreated as value types in SysML. Most ontological relationships have a system modeling counterpart (e.g., the "is a" relationship in OWL is comparable to "generalization" in SysML). Both ontology and SysML have provisions for creating instances of classes and blocks respectively.

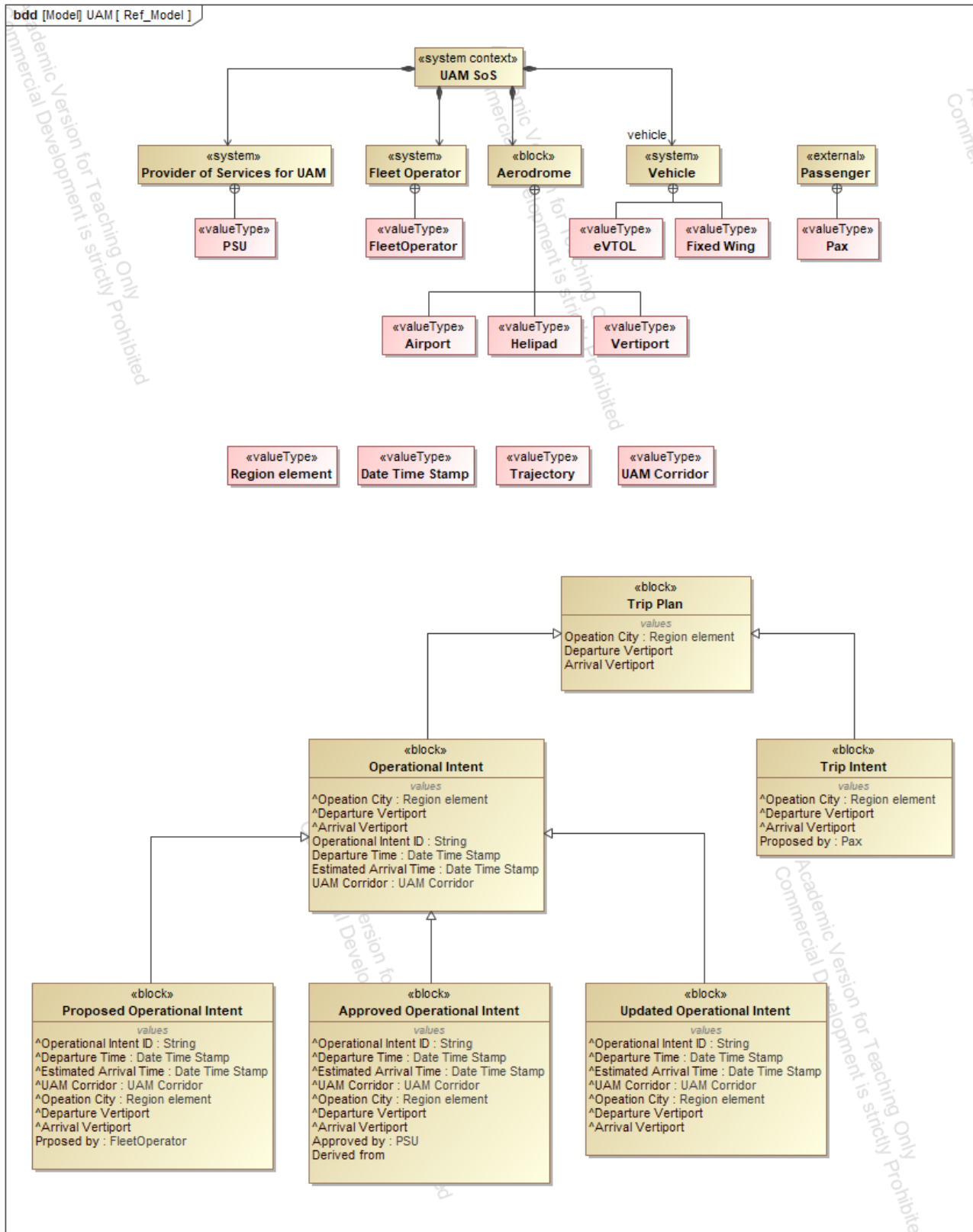


Figure 3 – Block Definition Diagram (BDD) of the reference model derived from the UAM ontology. The BDD shows the decomposition of the UAM SoS into its constituent systems (top), defines unique data types (pink blocks labeled as "«valueType»"), and shows the inheritance of *Operational Intent* and its subtypes from *Trip Plan*.

The reference model is intentionally designed to be sparse and abstract in order to keep it generic. The specific model, however, is intended for the development of the unique UAM architecture of a metropolitan area and therefore must include all the necessary details. The reference model defines the system blocks, interfaces, data types, and behavioral diagrams (e.g., activity diagrams, state machine diagrams, sequence diagrams, etc.). The specific model adds details to the ‘templates’ and instantiates the system blocks to represent the actual actors. Data types and communication protocols may be customized for each metropolitan area to ensure appropriate applicability.

The specific model serves as the main systems engineering artifact in the planning and development phase of a city or metro area’s UAM system. It provides the foundation for the operational model, which can be used for the real-time simulation and predictive analysis of the UAM system, or for testing various use cases and operational scenarios. For example, the specific UAM architecture model for New York City is the building block for an operational model that can simulate nominal airspace conditions at 8 AM on one day, and a weather emergency at 5 PM on another day. Additionally, the operational model needs to handle several databases from different sources. Ontology can help manage these databases and integrate them with UAM processes such as scheduling flight take-offs at a vertiport or maintaining safe operations during unexpected events.

4.3 Examples: Combined Use of Ontology and MBSE for UAM Applications

In this section, we demonstrate parts of the UAM modeling pipeline proposed in Section 4. through three examples. To highlight the diversity of applications of our proposed approach, we look at a different facet of the UAM system of systems in each example.

- Our first example focuses on the **fleet operator**, a key **actor** in the UAM SoS that executes several essential functions and acts as a liaison between the vehicles and the service providers.
- The second example demonstrates the use of ontology and MBSE for developing the **operational intent**. The information contained in it guides several UAM operations, thereby making it a critical **interface** in the system.
- The third example shows how ontology can aid **flight database query**, an important **process** that enables smooth UAM operations.

4.3.1 Fleet Operator

A UAM fleet operator owns and operates a fleet of aircraft and acts as a liaison between the vehicles and PSUs. It also interacts with the passengers to manage trip-related activities. Therefore, the primary responsibilities of the fleet operator include fleet management, passenger trip request handling, and end-to-end flight operations (scheduling, generation of operational intent, flight tracking, etc.).

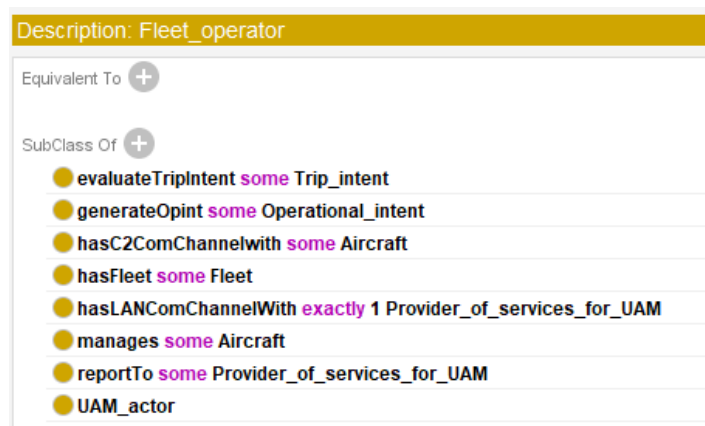


Figure 4 – *Fleet operator* defined as an object in ontology. The properties of this object reflect the functions performed by the fleet operator.

In the UAM ontology, actors like fleet operators can be defined as objects. The properties of the object specify the salient features of the actor. For example, Figure 4 shows the properties of the *Fleet operator* object. It specifies that the fleet operator owns a fleet, manages the aircraft in the fleet, has communication channels with aircraft and PSU, and executes functions such as trip intent evaluation, operational intent generation, etc.

While the ontology object provides a guideline for the fleet operator design, its behavior is developed through behavioral diagrams in SysML. Figures 5 and 6 show a few examples. In the reference model, the actor can be defined as a block (as seen earlier in Figure 3) and then its behavior can be developed through state machines, activity diagrams, etc. In our model, the high-level functions of the fleet operator are described using an iterative activity diagram, as shown in Figure 5. The nominal activities of the fleet operator are depicted here, including trip request evaluation, generation of operational intent and negotiation with the PSU for its approval, monitoring vehicles throughout their flight, mitigation of off-nominal events, and ensuring safe landing. The fork symbol on the gray action blocks indicates that they have an embedded diagram where the associated steps are modeled in detail.

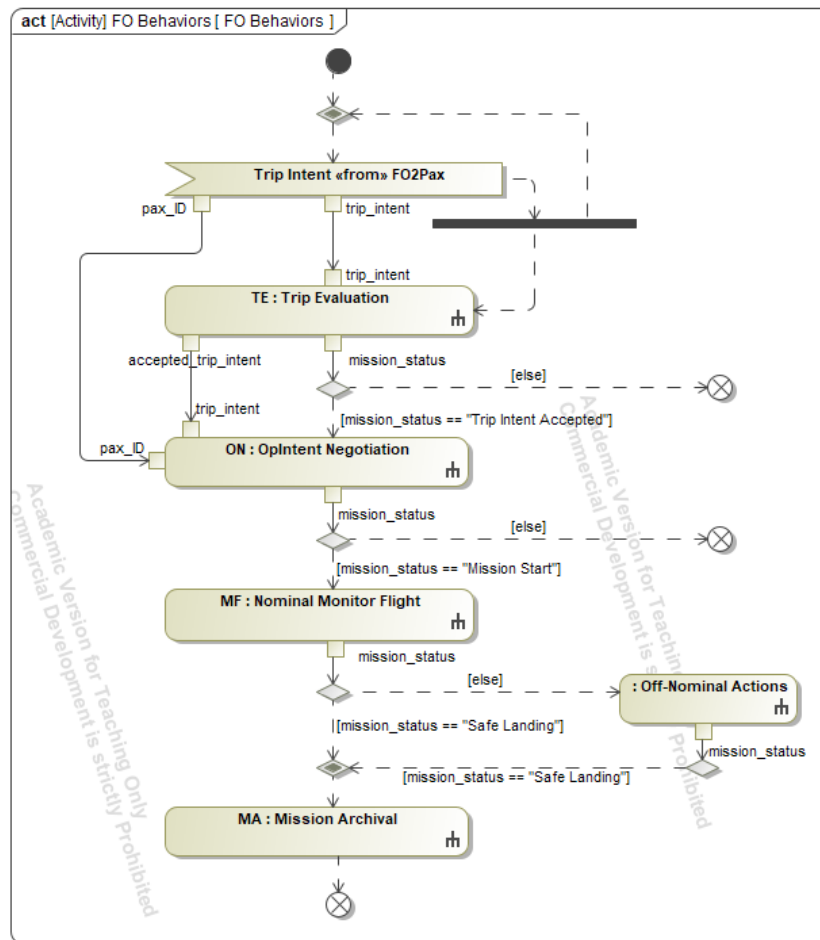


Figure 5 – High-level behavior of the fleet operator, expressed as an iterative activity diagram. The gray blocks represent actions, and the fork symbol indicates that they have an embedded detailed diagram.

For example, the embedded diagram for the “TE: Trip Evaluation” block is shown in Figure 6. When a passenger sends a trip request, the origin, destination, departure time, and other details are sent to the fleet operator as a ‘trip intent.’ The fleet operator determines if the trip can be feasibly undertaken by any aircraft in its fleet. If the trip intent is found to be acceptable, the next step is to generate an operational intent. However, if the fleet operator can not accept the trip, a rejection signal is sent to the passenger requesting the trip.

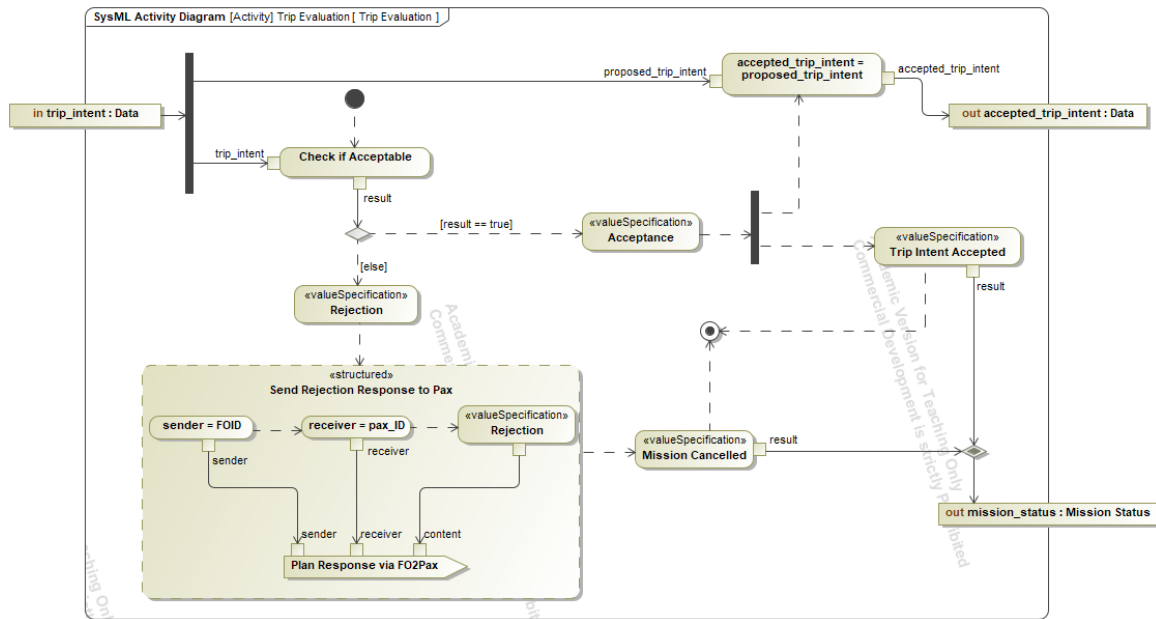


Figure 6 – Embedded activity diagram for the “TE: Trip Evaluation” action block in Figure 5; this diagram identifies the actions to be taken when the passenger’s trip request is accepted or rejected.

4.3.2 Operational Intent

An operational intent (also known as flight intent) is essentially the flight plan for a passenger trip within the UAM environment. It contains information like origin and destination, arrival and departure times, intended trajectory, etc. Different users or stakeholders may have a different understanding of the content and structure of an operational intent. The definition and data format can be formalized through the use of ontology, thereby ensuring uniformity across the entire UAM network. Figure 7 illustrates the inheritance relationships of the trip plan, trip intent, and operational intent. The *Trip plan* class includes trip intent proposed by the passenger and operational intent initiated by the fleet operator. Under the category of operational intent, there are three versions that indicate its different processing states – the fleet operator first proposes an operational intent, which is then ratified and approved by the PSU. If a change is required mid-flight, an updated operational intent is filed.

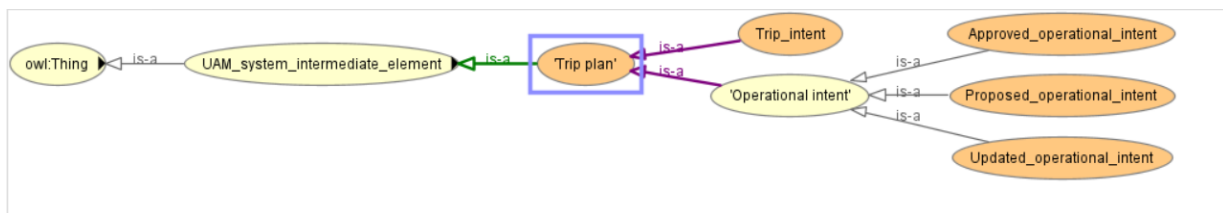


Figure 7 – Inheritance of *Operational Intent* and its children classes from the *Trip plan* class.

Figure 8 shows the class properties and the inheritance process related to *Operational Intent*. *Trip Plan* is the parent class of *Operational Intent*. This implies that *Operational Intent* inherits all the properties of *Trip Plan* and can additionally have its own unique set of properties. In SysML terminology, *Trip Plan* is a ‘generalization’ of *Operational Intent*. The mapping of the OWL class *Operational Intent* to the corresponding SysML block has been illustrated in Figure 9.

The Operational Intent block from the reference model can be customized for use in metro-area-specific architecture models and operational use cases. Let us consider two cities – New York City and Atlanta, Georgia. The two locations will have distinct UAM architectures (e.g., different vertiports, fleet operators, etc.). Therefore, each metropolitan area has its own version of the *Operational Intent* block within their respective specific models. For each trip in these two cities, an instance of the block can be created and values can be assigned to the attributes to create unique operational intents.

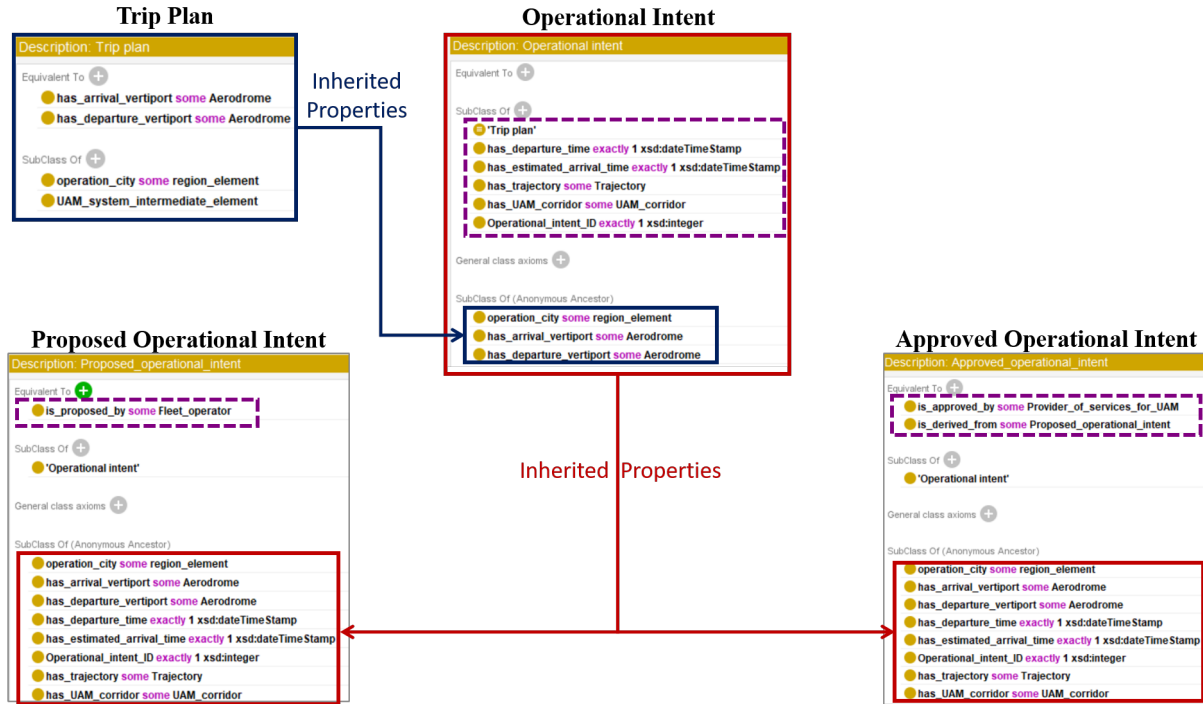


Figure 8 – Description the classes *Trip plan*, *Operational intent*, *Proposed_operational_intent*, and *Approved_operational_intent*. *Operational intent* inherits the properties of its parent, *Trip plan*, and also has additional properties such as departure time, trajectory, etc. Similarly, *Proposed_operational_intent*, and *Approved_operational_intent* inherit properties from *Operational intent* with a few additional properties of their own. The dashed purple boxes highlight the new properties of each class.

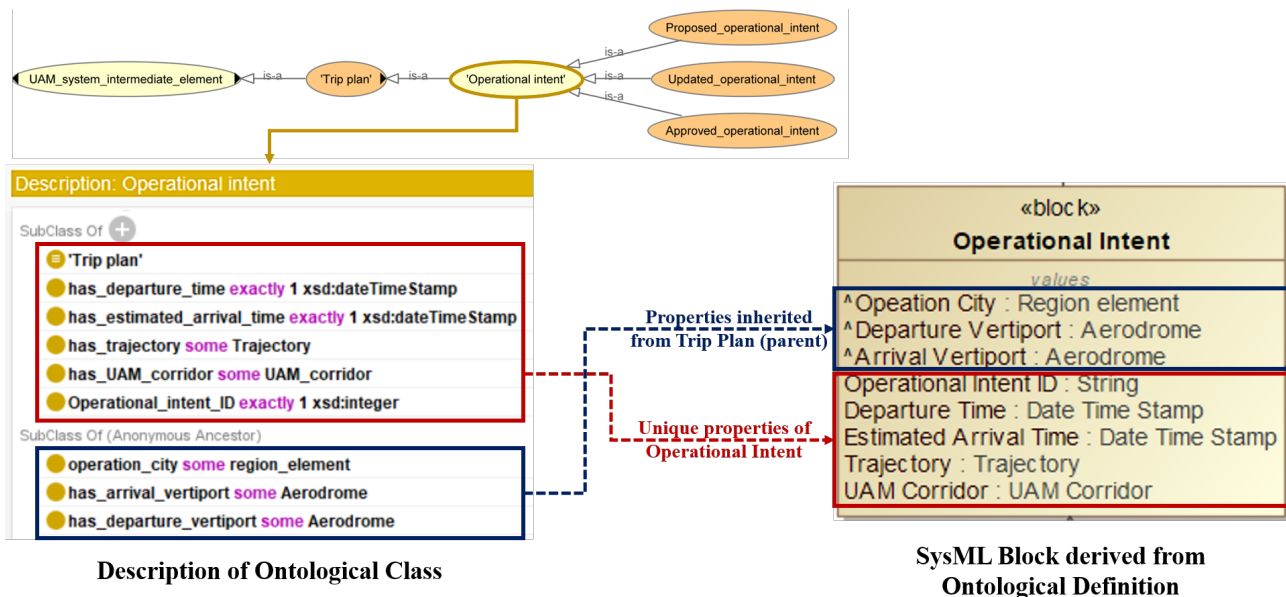


Figure 9 – Derivation of the reference SysML block for *Operational Intent* from its ontological definition. The properties inherited from the parent block (*Trip Plan*) are marked by the circumflex symbol (^) next to the property name in the SysML block and highlighted in a blue box. The properties in red boxes are unique to *Operational Intent*.

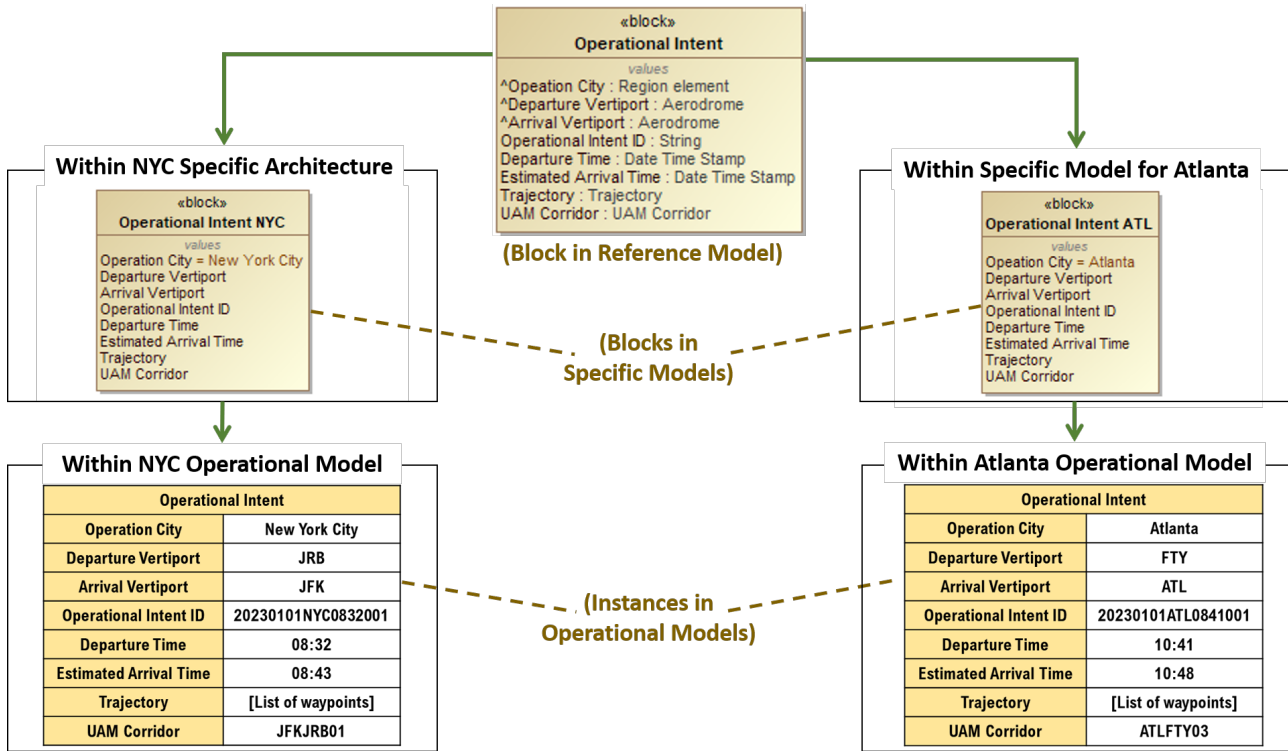


Figure 10 – SysML representation of the Operational Intent blocks in the specific models, along with instances for simulating use cases in the operational model. Note that the blocks in the two specific models are both derived from the reference model, but they have been customized to include the default values of the operation city.

This is illustrated in Figure 10. The basic structure of the *Operational Intent* class is derived from the reference architecture but a limited degree of customization can be applied to the corresponding blocks in the specific model. For example, the default value of operation city can be assigned as New York City and Atlanta respectively in *Operational Intent NYC* and *Operational Intent ATL*. Now, instances of these blocks can be created in the operational model to add all the details corresponding to these trips. This facilitates the simulation of different use cases at different times of the day and under a variety of operational conditions (e.g., adverse weather, high demand, etc.).

4.3.3 Flight Database Queries

The UAM air traffic management system is expected to utilize a large amount of data from various sources, which is hard to integrate using a relational database. The UAM SoS will require automatic and real-time decision-making using information across different data sources. Ontology-based queries can be useful in this regard. One may see ontology as a kind of graph database that can capture relationships across multi-domain data and make queries much faster than traditional relational databases, especially when the request involves references to or combinations of multiple databases. NASA's "Air Traffic Management Ontology" (ATMonto) [24, 25] has demonstrated the ability to integrate multiple data sources to assist air traffic management and analysis for traditional aviation. In building the UAM ontology, we have repurposed parts of ATMonto and added new capabilities that are unique to UAM. In this section, we demonstrate the use of queries in ontology – specifically a SPARQL query – for air traffic management under adverse weather conditions.

SPARQL stands for SPARQL Protocol and RDF Query Language. It is similar to SQL (Structured Query Language) in relational databases. However, SPARQL is designed to query RDF (Resource Description Framework) data which is stored in the triple (subject-predicate-object) format. SPARQL can utilize multiple sources of information for a single query and is able to work with both classes and instances. In our example, we look at the scenario illustrated in Figure 11. We consider an operational UAM SoS where multiple databases are maintained by various actors (e.g., the fleet operator

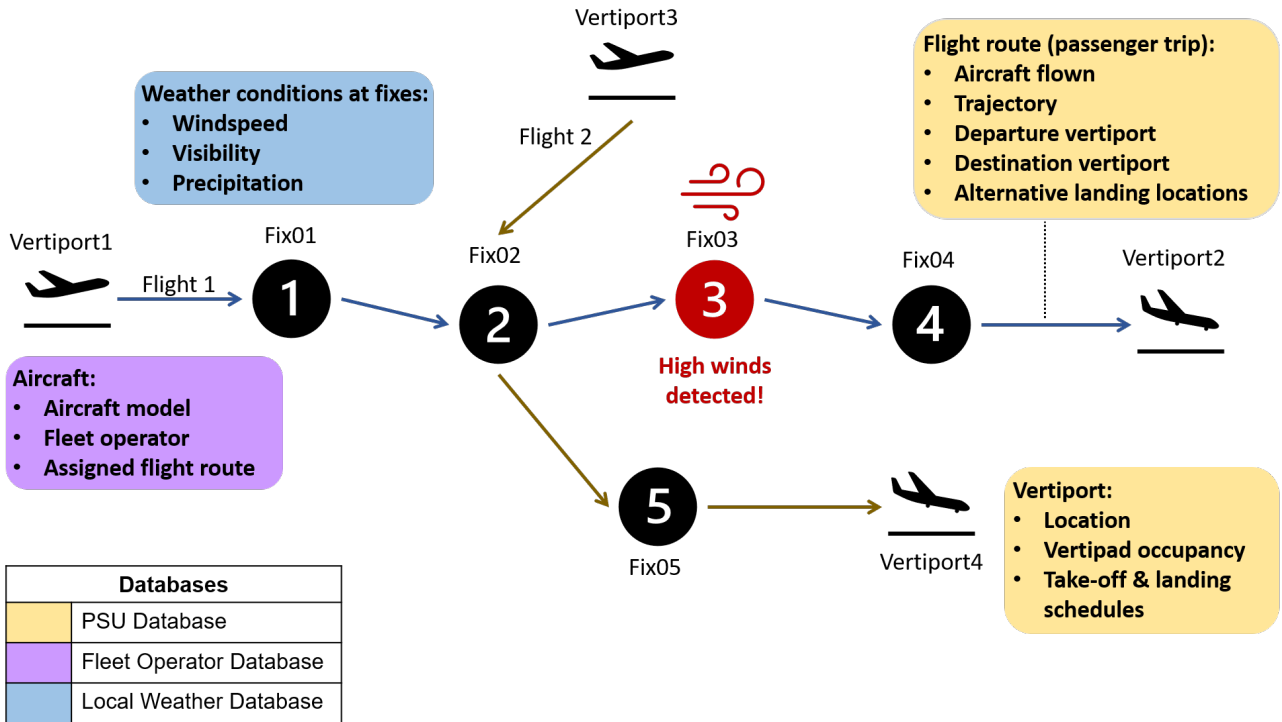


Figure 11 – Schematic representation of the high winds example. UAM flights have planned routes that pass through designated fixes. The PSU tracks trip information and vertiport status, the fleet operator maintains its fleet database, and the current weather for each of the fixes can be retrieved from the local weather database. If high winds are detected at the location of Fix03, then the flight routes passing through it must be identified and the appropriate actors must be notified.

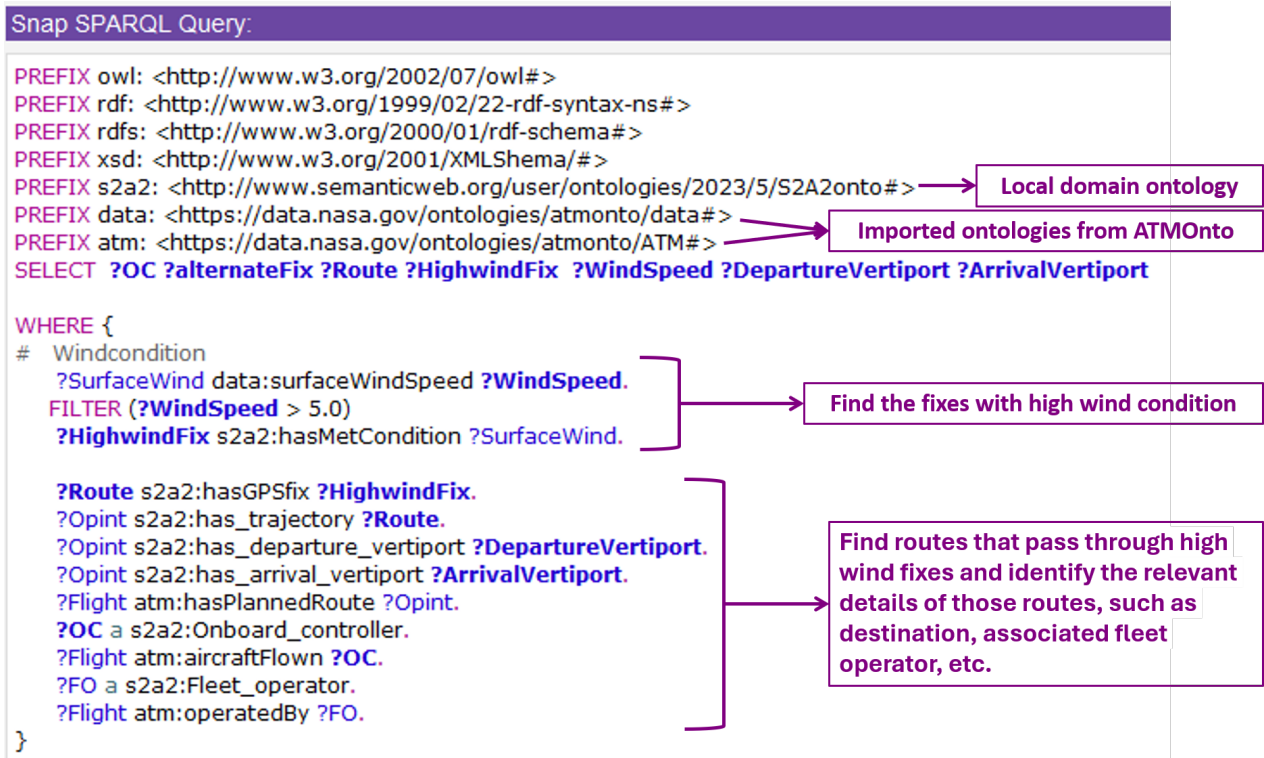


Figure 12 – SPARQL query to identify details of the flight routes passing through a high wind location. We reused some of the ontologies from ATMOnto.

maintains a fleet vehicle database, the PSU keeps track of active flights and vertiport conditions, and weather data for the fixes or waypoints is retrieved from the local weather database). If adverse weather, such as high winds, is detected at any of the fixes, all the flights passing through it must be warned. This action can be performed by crafting a SPARQL query that can quickly identify the affected routes, notify the associated fleet operators, send automated alerts to the vehicles, and ensure safe passage or a diversion to an alternative route, if needed. These capabilities lead to heightened situational awareness and facilitate effective air traffic management. Our SPARQL query is depicted in Figure 12, and the results are shown in Figure 13.

?OC	?Route	?HighwindFix	?WindSpeed	?DepartureVertiport	?ArrivalVertiport
s2a2:OC1	s2a2:Trajectory1	s2a2:GPSfix1	3.1	s2a2:Vertiport1	s2a2:Vertiport2
s2a2:OC1	s2a2:Trajectory1	s2a2:GPSfix2	2.1	s2a2:Vertiport1	s2a2:Vertiport2
s2a2:OC1	s2a2:Trajectory1	s2a2:GPSfix3	7.2	s2a2:Vertiport1	s2a2:Vertiport2
s2a2:OC1	s2a2:Trajectory1	s2a2:GPSfix4	0.8	s2a2:Vertiport1	s2a2:Vertiport2
s2a2:OC2	s2a2:Trajectory2	s2a2:GPSfix2	2.1	s2a2:Vertiport3	s2a2:Vertiport2
s2a2:OC2	s2a2:Trajectory2	s2a2:GPSfix5	1.6	s2a2:Vertiport3	s2a2:Vertiport2

(a) Query result for all the flight routes and the weather conditions.

?OC	?Route	?HighwindFix	?WindSpeed	?DepartureVertiport	?ArrivalVertiport
s2a2:OC1	s2a2:Trajectory1	s2a2:GPSfix3	7.2	s2a2:Vertiport1	s2a2:Vertiport2

(b) Route information for the aircraft entering a high wind location.

Figure 13 – Query results for the scenario described in Figure 11.

This mock-up example demonstrates how ontology can integrate multiple data sources and act as a graph database to perform high-speed, automatic querying for UAM operations. These features of ontology can be used in conjunction with the operational model. While this is a simple case for which a relational database may have sufficed, ontology-based methods have an advantage when the query becomes more complex and spans more data sources. The ability to relate multiple data sources also enables potential applications in data mining and intelligent systems, which could further enhance the UAM air traffic management system's analytical capabilities.

5. Merits and Limitations

Developing an ontology-MBSE pipeline for UAM applications offers significant advantages, such as establishing a reusable, modular, foundational scheme for modeling UAM operations, promoting standardization, and facilitating communication among stakeholders. The combined use of ontology and MBSE can (i) help establish a unanimous understanding of all the UAM systems and processes, (ii) ensure universally understandable and applicable models and documentation for future developments, and (iii) facilitate policy-making, establishment of standards, and international cooperation.

Additionally, our approach eases the challenges associated with modeling language intricacies and semantics, which enables a more focused exploration of UAM applications. While existing architecture frameworks like DoDAF and NAF offer assistance in the modeling process, they lack specific UAM considerations. Our approach provides a UAM-specific 'blueprint' for architecture creation, enhancing modularity and reusability while providing the niche UAM context. There is immense worldwide potential in this field, as evidenced by the FAA's collaboration with Korea in advanced air mobility [26], European initiatives such as the HorizonUAM project [27, 28] at the German Aerospace Center (DLR), and the growing UAM market in the Asia-Pacific region [29, 30]. This foundational work aims to facilitate such efforts by enhancing coordination and mutual understanding within the UAM community.

The merits come with inherent limitations shared by MBSE and ontology modeling. Both processes can be time-consuming and labor-intensive [31]. The modeling tools' flexibility can make modeling complex systems less straightforward, potentially leading to a reluctance among users to embrace

this approach. However, with some dedication, these models can be highly efficient and beneficial once established, compensating for the initial investment in time and resources. Another limitation includes the challenge of unifying ideas and concepts within the UAM domain. UAM is a developing concept and an area of active research. Therefore, the development of complete, comprehensive, and inclusive UAM ontological and MBSE models requires significant time, effort, and collaboration among the various multidisciplinary teams.

6. Conclusion

UAM represents a transformative concept for future urban transportation. MBSE and ontological modeling can play a crucial role in the rapid modeling of UAM use cases using a common set of semantics and structures. MBSE provides a structured approach to conceptualize, analyze, and design the UAM SoS, while ontological modeling helps establish a common language and vocabulary for stakeholders to communicate effectively. Our proposed model-building pipeline combines ontology, reference models, metro-area-specific architectures, and operational models to comprehensively represent the UAM SoS. Starting with a UAM ontology, we derive a reference model that captures the high-level structure and relationships within the domain. Specific UAM architectures can be rapidly modeled from the baseline architecture (reference model) and then used for simulating various operational use cases. This allows for a detailed exploration of UAM operations, including emergent behaviors. This work represents an ongoing effort; our research continues to expand the ontology and the reference model, fine-tuning the pipeline, and testing several UAM use cases. Since the dynamic evolution of the UAM concepts may necessitate frequent revisions to reflect changing requirements and technologies, we progress in our work while being mindful of the required level of flexibility and adaptability in our methods.

7. Contact Author Email Address

Contact author: Sonali Sinha Roy

Email: ssinhar@purdue.edu

8. Copyright Statement

The authors confirm that they, and/or their company or organization, hold copyright on all of the original material included in this paper. The authors also confirm that they have obtained permission, from the copyright holder of any third party material included in this paper, to publish it as part of their paper. The authors confirm that they give permission, or have obtained permission from the copyright holder of this paper, for the publication and distribution of this paper as part of the ICAS proceedings or as individual off-prints from the proceedings.

References

- [1] Urban air mobility (UAM) concept of operations 2.0. Technical report, Federal Aviation Administration, 2023. Accessed: 2023-12-18.
- [2] I Levitt, N Phojanamongkolkij, A Horn, and K Witzberger. UAM research roadmap rev 1.2. Technical report, NASA/TM-20220008917, Washington, DC, 2022.
- [3] Daniel DeLaurentis. Understanding transportation as a system-of-systems design problem. In *43rd AIAA aerospace sciences meeting and exhibit*, page 123, 2005.
- [4] Prajwal Shiva Prakasha, Nabih Naeem, Patrick Ratei, and Björn Nagel. Aircraft architecture and fleet assessment framework for urban air mobility using a system of systems approach. *Aerospace Science and Technology*, 125:107072, 2022.
- [5] Sai Mudumba, Hsun Chao, Brady J Beck, Sonali Sinha Roy, Tien-Yueh Fung, Jack L Wiley, and Daniel A DeLaurentis. Model-based systems engineering approach for simulating uml-5 uam operations. In *AIAA AVIATION 2022 Forum*, page 4075, 2022.
- [6] Brian P. Hill, Dwight DeCarme, Matt Metcalfe, Christine Griffin, Sterling Wiggins, Chris Metts, Bill Bastedo, Michael D. Patterson, and Nancy L. Mendonca. UAM vision concept of operations (conops) UAM maturity level (UML) 4. Technical report, NASA, 2020.
- [7] Virginia L Stouffer, William B Cotton, Randall A DeAngelis, Danial M Devasirvatham, Thomas B Irvine, Richard E Jennings, Ronald D Lehmer, Thanh C Nguyen, Michelle A Shaver, and Casey Joseph Bakula. Reliable, secure, and scalable communications, navigation, and surveillance (cns) options for urban air mobility (uam). Technical report, SAIC, 2020.

- [8] UAS Northeast. Advanced air mobility (aam) vertiport automation trade study. Technical report, Northeast UAS Airspace Integration Research Alliance (NUAIR), 2020.
- [9] Carlos C Insaurralde and Erik Blasch. Ontologies in aeronautics. In *Advances in Aeronautical Informatics: Technologies towards Flight 4.0*, pages 67–85. Springer, 2018.
- [10] Erik Blasch, Ali K Raz, Roberto Sabatini, and Carlos C Insaurralde. Information fusion as an autonomy enabler for uas traffic management (utm). In *Proceedings of the AIAA Scitech Forum*, pages 1–12, 2021.
- [11] Ludvig Knöös Franzén. *A System of Systems View in Early Product Development - An Ontology-Based Approach*. Linköping University Electronic Press, 05 2023.
- [12] Ludvig Knöös Franzén. *An Ontological and Reasoning Approach to System of Systems*. PhD thesis, Linköping University Electronic Press, 2021.
- [13] Ludvig Knöös Franzén, Ingo Staack, Petter Krus, Christopher Jouannet, and Kristian Amadori. A breakdown of system of systems needs using architecture frameworks, ontologies and description logic reasoning. *Aerospace*, 8(4):118, 2021.
- [14] Caitlyn Singam and Jeffrey Carter. Model-based systems engineering (MBSE). [https://sebokwiki.org/wiki/Model-Based_Systems_Engineering_\(MBSE\)](https://sebokwiki.org/wiki/Model-Based_Systems_Engineering_(MBSE)), November 2023.
- [15] Michael Sievers. Semantics, metamodels, and ontologies. In *Handbook of Model-Based Systems Engineering*, pages 1–32. Springer, 2022.
- [16] Azad M. Madni and Michael Sievers. Model-based systems engineering: Motivation, current status, and research opportunities. *Systems Engineering*, 21(3):172–190, 2018.
- [17] Robert Cloutier, Gerrit Muller, Dinesh Verma, Roshanak Nilchiani, Eirik Hole, and Mary Bone. The concept of reference architectures. *Systems Engineering*, 13(1):14–27, 2010.
- [18] S. R. Kelly. A reference architecture for rapid cubesat development, 2021.
- [19] Jim Murphy. NASA's MBSE Approach for Advanced Air Mobility. <https://ntrs.nasa.gov/citations/20220003085>, 2022.
- [20] Nicolas Jäckel, Christian Granrath, Robert Schaller, Malte Grotenrath, Maximilian Fischer, Philipp Orth, and Jakob Andert. Integration of vtol air-taxis into an existing infrastructure with the use of the model-based system engineering (mbse) concept cube. In *Vertical Flight Society 76th Annual Forum and Technology Display*, 2020.
- [21] LC Van Ruijven. Ontology for systems engineering as a base for mbse. In *INCOSE International Symposium*, volume 25, pages 250–265. Wiley Online Library, 2015.
- [22] Uri Shani, Shmuela Jacobs, Niva Wengrowicz, Dov Dori, and N Wengrowiz. Engaging ontologies to break mbse tools boundaries through semantic mediation. In *Conference on Systems Engineering Research*, pages 1–10, 2016.
- [23] Dominique Ernadote. An ontology mindset for system engineering. In *2015 IEEE International Symposium on Systems Engineering (ISSE)*, pages 454–460. IEEE, 2015.
- [24] Richard M. Keller. Ontologies for aviation data management. In *2016 IEEE/AIAA 35th Digital Avionics Systems Conference (DASC)*, pages 1–9, 2016.
- [25] Richard M. Keller. Building a knowledge graph for the air traffic management community. In *Companion Proceedings of The 2019 World Wide Web Conference, WWW '19*, page 700–704, New York, NY, USA, 2019. Association for Computing Machinery.
- [26] FAA. US FAA, Korea to Partner on Advanced Air Mobility Certification, Operations, 2023. Accessed on 2023-12-14.
- [27] Bianca I Schuchardt, Dennis Becker, Richard-Gregor Becker, Albert End, Thomas Gerz, Frank Meller, Isabel C Metz, Malte Niklaß, Henry Pak, Sebastian Schier-Morgenthal, et al. Urban air mobility research at the dlr german aerospace center—getting the horizonuam project started. In *AIAA Aviation 2021 Forum*, page 3197, 2021.
- [28] Lukas Asmer, Henry Pak, Prajwal S Prakasha, Bianca I Schuchardt, Peter Weiland, Frank Meller, Christoph Torens, Dennis Becker, Chen Zhu, Karolin Schweiger, et al. Urban air mobility use cases, missions and technology scenarios for the horizonuam project. In *AIAA Aviation 2021 Forum*, page 3198, 2021.
- [29] Kate Sarsfield. Is Asia-Pacific where urban air mobility revolution starts? *Flight Global*, Feb 2022.
- [30] Chris Kjellaard. Thales sees Asia-Pac as fertile ground for UAM development. *Aviation International News*, May 2024.
- [31] S Subarna, Ankita K Jawale, Akshay S Vidap, Subodh D Sadachar, Shana Fliginger, and Srimannarayana Myla. Using a model based systems engineering approach for aerospace system requirements management. In *2020 AIAA/IEEE 39th Digital Avionics Systems Conference (DASC)*, pages 1–8. IEEE, 2020.